MSML606 | University of Maryland | Spring 2026

Plagiarism Detection using Hash Maps

Rabin-Karp Rolling Hashes + Jaccard Similarity

Universities | Research Integrity | Academic Publishing | Legal Documents

Anoushka Anand · FNU Hardik · Muazuddin Syed

Link to the Repo

The Problem

Plagiarism detection is used every day. Universities check student essays, publishers verify manuscripts, and companies protect intellectual property.

📄 Scale

Thousands of documents submitted at once. Manual review is impossible.

Real case: A university receives 5,000 essays per semester.

🔁 Paraphrasing

Plagiarists rearrange sentences, not just copy-paste.

Simple word-match tools miss reworded content entirely.

⚡ Speed

Checking every pair of documents naively takes millions of comparisons.

With 10k docs: 50 million pairs to check. Too slow.

Our Solution: How It Works

In simple terms: we convert text into numeric fingerprints, store them in a lookup table, and check how much two documents' fingerprints overlap.

1

Preprocess

Lowercase text, remove punctuation, normalize whitespace so noise doesn't affect matching.

2

Hash K-grams

Slide a window of k words across the text. Convert each phrase into a number (hash) in O(1) per step.

3

Index Lookup

Check each hash against the inverted index: hash to set of doc IDs. Skip all non-matching docs.

4

Score & Flag

Compute Jaccard overlap. Flag any document scoring above the similarity threshold.

The Dataset

Meets 10,000 item requirement

2,400+
Document Chunks
150 words each

800k+
Unique Hash Keys
Inverted index entries

3
Source Books
Project Gutenberg

5
K-gram Size
Default window

Source books (public domain, no copyright issues):

Moby Dick

Herman Melville

Seafaring epic with rich, varied vocabulary

Pride and Prejudice

Jane Austen

Social drama with dialogue-heavy text

Alice in Wonderland

Lewis Carroll

Fantasy fiction with unusual word patterns

Why Books? And Does This Work on Real Essays?

Fair question. The demo uses book chunks as a stand-in corpus. Here is the honest explanation of what the tool actually does, and how it applies to real student essays.

What the tool actually does

The corpus is just a list of reference documents.

To check 10,000 student essays against each other, you would:

1. Load all 10,000 essays into the corpus CSV.
2. Build the hash index from those essays.
3. Run each essay as a query against that index.

The algorithm does not care whether the documents are books or essays. It works on any text.

Why we used books for this demo

Publicly available

Project Gutenberg books are free, public domain, and require no sign-ups or permissions.

Ground truth is known

We know exactly which chunks were copied because we generated the copies ourselves. This lets us verify the algorithm works.

No real student data

Using actual student essays would raise privacy concerns. Books let us demonstrate the same algorithm safely.

Same algorithm, any corpus

Swap the CSV with real essays and the detection pipeline runs identically.

Algorithm Deep Dive

Plain English: Every phrase in a document gets converted to a number. We store a lookup table of those numbers. To compare two documents, we just count how many numbers they share.

Rabin-Karp Rolling Hash

A k-gram is a window of k consecutive words.
For 150 words with k=5: 146 overlapping k-grams.

Rolling hash formula:
$h = \Sigma \text{ ord}(\text{char}[i]) \times P^i \mod M$

P = 31 | M = 1,000,003

Key insight: each step slides the window in O(1) with no rehashing from scratch.
Total cost: O(L) per document, regardless of k.

Jaccard Similarity + Inverted Index

Inverted index:
{ hash to set of doc_IDs }

Only documents sharing at least one hash are ever scored.
Everything else is skipped.

Jaccard score:
$J = |shared| / (|Q| + |D| - |shared|)$

Score of 1.0 = identical.
Score of 0.0 = no overlap.

Docs above the threshold are flagged as plagiarised.

Handling Real-World Noise

Real text data is messy. Here is how the pipeline handles each issue before a single hash is computed.

Aa

Mixed Case

Lowercased so "The" and "the" hash identically.

!?

Punctuation

"hello," and "hello" are the same token after stripping.

Extra Whitespace

Normalized to single spaces before windowing.

∅

Empty / Short Docs

Rows with no text skipped. Docs shorter than k produce no k-grams.

≈

Hash Collisions

With M = 1,000,003 and typical doc sizes, false positive rate is negligible.

?

Missing Fields

Missing CSV metadata loaded with defaults; processing continues.

Performance at Scale

1,500,000

Comparisons with the naive approach on 10k docs

~1,000

Operations with the hash map approach on same corpus

O(L)

Query time, independent of corpus size

Complexity Summary	
Index Build	**O(N × L)**
Query Hash	**O(L)**
Candidate Score	**O(matches)**
Total Query	**O(L + matches)**

Demo: What the Output Looks Like

Setup (3 commands, no pip install):

01
```
$ git clone
https://github.com/hardikkamboj/plagirism-detecti
on-hash-maps.git
```

02
```
$ python generate_corpus.py
```

03
```
$ python detector.py
```

Sample output for a plagiarised passage:

Found 1 matching document(s)

pride_and_prejudice_0331	Score:	Source:
	43.75%	pride_and_prejudice

Low match

What does score 43.75%?

43.75% of the query's k-gram fingerprints matched this document. That is strong evidence of copying. Anything above the threshold (default 0.10) is flagged automatically.

Key Takeaways

- Hash maps turn a 1.5M-comparison problem into ~1,000 operations per query
- Rolling hashes compute every k-gram fingerprint in O(1) — no redundant work
- Jaccard similarity catches paraphrased content, not just exact copy-paste
- Preprocessing ensures noisy, real-world text works reliably with the algorithm

Thank You

Anoushka Anand · FNU Hardik · Muazuddin Syed